Production-Grade Two-Week Take-Home Project Specification

Title

Full-Stack Marketplace Platform

Objective

Design and build a **production-style marketplace platform** that includes: - a **Flutter mobile application** for end users, - a **FastAPI backend** with well-structured REST APIs, - a **MySQL database** with strong relational design, - and an **admin panel** for moderation, operations, analytics, and platform control.

This project is intended for **top-tier, carefully selected students**. It is not a toy assignment. The submission must reflect the thinking and execution quality expected from a junior engineer who can soon become production-capable.

The platform must feel like a real startup product that could plausibly be launched with further polishing.

---

1. Product Scenario

Students must build a marketplace-style system in which users can: - register and authenticate, - create, edit, and manage posts/listings, - upload and manage listing media, - browse and search listings, - filter and sort listings, - view public owner/seller information, - view all listings of a particular owner, - save favorite listings, - message other users, - send attachments inside messages, - report listings or users, - manage language preferences, - pay for platform features, - purchase promotions such as targeting/boosting, - view notifications, - and manage their account/profile.

Administrators must be able to: - manage users, - manage categories and attributes, - moderate listings, - inspect conversations when needed for abuse handling, - review reports, - manage payments, - manage promotions, - manage featured/boosted content, - manage language content, - monitor analytics and operational dashboards, - and review audit trails.

The platform may be themed as one of the following: - car marketplace, - real estate marketplace, - service marketplace, - electronics marketplace, - general classified marketplace.

Students may choose the domain, but the feature set must remain complete and coherent.

---

2. Timeframe and Intent

Duration

14 days

Goal of the assignment

This assignment measures: - engineering maturity, - architecture quality, - execution under time constraints, - product sense, - code cleanliness, - backend discipline, - database modeling, - mobile UI/UX quality, - and ability to build an end-to-end system.

This is intentionally ambitious. Reviewers do not expect every advanced feature to be perfect, but they do expect the student to: - choose a correct architecture, - implement the important flows fully, - and make intelligent trade-offs.

A submission should feel like **“a serious MVP built by a strong engineer”**, not a classroom demo.

---

3. Required Technology Stack

3.1 Mobile Application

Required: - Flutter - Dart - API-driven architecture - clear state management

Accepted state management options: - Riverpod - Bloc/Cubit - Provider - another structured option if justified

The mobile app must not be a pile of uncontrolled `setState` calls.

3.2 Backend

Required: - Python - FastAPI - Pydantic for validation - SQLAlchemy / SQLModel / equivalent ORM or structured DB layer - JWT-based authentication - layered project structure

3.3 Database

Required: - MySQL

Expected: - normalized relational design where reasonable, - correct foreign keys, - sensible indexing, - constraints, - migrations.

3.4 Admin Panel

Allowed implementations: - FastAPI templates / Jinja admin, - separate frontend, - lightweight web admin, - or a properly integrated admin framework.

The admin panel must be secure, functional, and operationally useful.

3.5 Deployment / Runtime

Minimum: - clear local setup - reproducible run instructions

Recommended: - Docker Compose - deployed backend - deployed admin panel - demo APK

---

4. Product Modules Overview

The system must include the following production-style modules:

1. Authentication and authorization
2. User profile and public user pages
3. Categories and listing taxonomy
4. Listing/post CRUD
5. Listing media management
6. Search, filtering, sorting, and pagination
7. Owner visibility and owner listing pages
8. Favorites / saved items
9. Messaging
10. Message attachments
11. Notifications
12. Reporting and moderation
13. Payments / wallet / transactions
14. Promotions / targeting / boosting
15. Languages / localization support
16. Admin operations panel
17. Analytics / dashboards
18. Audit logs
19. Security / permissions
20. Documentation and delivery quality

---

5. Roles and Access Control

5.1 Required roles

The system must support at minimum: - **guest** - **authenticated user** - **admin**

Recommended additional roles: - moderator - support - superadmin

5.2 Permission expectations

Guests may: - browse listings - search and filter - view listing detail - view owner public profile

Authenticated users may: - create and manage own listings - favorite listings - message others - upload attachments where allowed - purchase promotions - view own payments and transaction history - report listings/users - manage profile and language settings

Admins may: - moderate users and listings - review reports - inspect payment history - create/edit promotion packages - feature/unfeature/boost listings - review dashboards - manage categories - manage supported languages/content

Authorization checks must be enforced server-side.

---

6. Authentication and Authorization Requirements

6.1 User authentication

Required flows: - registration - login - logout - session persistence - token refresh or durable session strategy - forgot password flow - reset password flow - change password while logged in

6.2 Registration fields

At minimum: - full_name - email and/or phone - password - confirm password - optional language preference

6.3 Auth security requirements

Must include: - hashed passwords - JWT access token - secure token validation - route protection - role-based authorization - account status checks

Recommended: - refresh tokens - email/phone verification mock flow - login attempt controls

6.4 User statuses

Required statuses: - active - blocked/suspended - pending_verification - deleted/deactivated

If a user is suspended, business logic must restrict actions consistently.

---

7. User Profile and Public Owner Pages

7.1 Private profile

Each user must be able to: - view personal profile - edit profile fields - upload/change profile image - set language preference - manage contact preferences - view own listings - view favorite listings - view purchased promotions - view payment history - view message inbox - view sent reports

7.2 Profile fields

Suggested minimum fields: - id - full_name - email - phone - profile_image_url - bio - city - preferred_language - account_status - created_at - updated_at

Recommended additional fields: - last_seen_at - verified_badge - company_name - seller_type - response_rate

7.3 Public owner page

Every listing must show clearly who the owner/seller is.

The user must be able to tap/click the owner and open a public owner page containing: - owner display name - profile image - city - join date - number of active listings - all active listings by this owner - optional rating/verified badge if implemented

7.4 Owner listings page

The system must support: - endpoint to retrieve all public active listings by owner - mobile screen showing all listings of that owner - pagination - sort options

This is mandatory.

---

8. Categories, Taxonomy, and Dynamic Attributes

8.1 Categories

The system must support categories.

Minimum category fields: - id - name - slug/code - is_active - display_order - created_at

8.2 Optional subcategories

Recommended: - parent_category_id

8.3 Dynamic category attributes

For production-style readiness, the system should support category-specific attributes.

Examples: - cars: brand, model, year, mileage, fuel type - real estate: rooms, area, floor, building type - electronics: brand, condition, warranty

Minimum requirement: - at least demonstrate a strategy for category-specific fields

Acceptable implementations: - separate typed tables, - JSON attribute store with validation, - or a hybrid approach.

Students must explain the choice.

---

9. Listing / Post CRUD Requirements

9.1 Core listing concept

Listings/posts are the primary content object.

Every authenticated user must be able to: - create a listing - edit own listing - delete own listing - archive own listing - deactivate/reactivate own listing where permitted - mark listing as sold/closed where relevant

9.2 Listing fields

Required fields: - id - owner_id - category_id - title - description - price - currency - city/location - status - created_at - updated_at

Recommended additional fields: - condition - latitude - longitude - contact_preference - is_negotiable - view_count - promotion_status - moderation_status - published_at - expires_at

9.3 Listing statuses

At minimum: - draft - pending_review - approved/published - rejected - archived - inactive - sold/closed

The backend must enforce valid transitions.

9.4 Listing CRUD behavior

Create

- user submits listing form
- validation occurs
- media may be uploaded
- listing enters correct initial state

Update

- only owner or admin can update
- immutable vs mutable fields must be considered
- editing a published listing may optionally return it to pending moderation depending on rules

Delete

- hard delete is discouraged
- soft delete or archival strategy preferred

Read

- public feed only shows publicly visible listings
- owner can see own hidden or draft items in personal area

9.5 Validation rules

Examples: - title required and length constrained - description minimum length - price must be positive or zero depending on business rules - category required - owner required - status controlled by system - invalid media types rejected

9.6 Moderation rules

A listing must not become public if: - category invalid - owner suspended - moderation rejected - required fields missing

---

10. Listing Media Requirements

10.1 Image support

Listings must support multiple images.

Required behaviors: - upload multiple images - order images - set primary image - delete image - replace image

10.2 Media rules

Must validate: - file type - file size - image count limit

Recommended: - image compression/resizing - thumbnail generation

10.3 Storage approach

Can be: - local development storage, - cloud/object storage in design, - or abstracted file service.

Students should not hardcode broken file paths.

10.4 Optional video support

Optional but allowed.

---

11. Browse, Search, Filter, Sort, and Pagination

11.1 Browse feed

The mobile app must have a listings feed/home feed.

11.2 Listing details

Details screen must include: - title - price - description - media gallery - owner card - owner listings shortcut - category/location info - favorite button - message/contact button - report button - promoted/featured indicator if applicable

11.3 Search

Must support keyword search.

Expected search behavior: - title matching - description matching - optionally owner/category matching

11.4 Filters

Required filters: - category - city/location - min price - max price - status (for admin/owner contexts)

Recommended filters: - condition - date posted - promoted only - seller type - category-specific fields

11.5 Sorting

Required: - newest first - oldest first - price ascending - price descending

Recommended: - promoted first - most viewed

11.6 Pagination

Mandatory for list endpoints.

The backend must return pagination metadata, for example: - page - page_size - total_items - total_pages

Infinite scroll or paged UI is acceptable.

---

12. Favorites / Saved Listings

Users must be able to: - add listing to favorites - remove listing from favorites - view favorites list

Requirements: - same listing cannot be saved twice by same user - favorites on removed/archived listings must be handled gracefully - favorites screen must support pagination

---

13. Messaging System Requirements

Messaging is mandatory and is a major part of this project.

13.1 Messaging goals

Users must be able to communicate regarding a listing.

13.2 Required messaging model

Minimum required behavior: - user opens listing - taps message/contact owner - conversation is created or reopened - messages are exchanged between two users - conversation is linked to listing context

13.3 Messaging features

Required: - conversations list - conversation detail view - send text message - receive/read messages from API - show sender/receiver correctly - show timestamps - unread/read state - last message preview in inbox

Recommended: - conversation search - conversation pinning/muting - typing indicator mock state

13.4 Messaging business rules

- users cannot message themselves unless explicitly allowed
- blocked/suspended users may be restricted
- messages must belong to a conversation
- each conversation should be between participants with optional listing context

13.5 Conversation fields

Suggested: - id - listing_id - created_by_user_id - participant_a_id - participant_b_id - last_message_at - created_at - updated_at

13.6 Message fields

Suggested: - id - conversation_id - sender_id - message_type - text_body - is_read - sent_at - edited_at - deleted_at (optional soft delete)

---

14. Message Attachments Requirements

Attachments in messaging are mandatory.

14.1 Allowed attachment types

At minimum support: - images - documents such as PDF

Optional: - audio - additional file types

14.2 Required behaviors

Users must be able to: - attach one or more files to a message, or at minimum one file per message depending on chosen scope - view attachment previews where appropriate - download/open attachments - send messages with attachment only, text only, or text + attachment

14.3 Attachment validation

Must validate: - allowed file types - file size limits - file count limits - secure file path / storage reference

14.4 Data model suggestion

Attachment fields may include: - id - message_id - file_name - original_name - mime_type - file_size - file_url/path - created_at

14.5 Security expectations

- attachment access must be permission-aware
- do not expose arbitrary file system access
- sanitize filenames or replace with generated names

---

15. Notifications Requirements

The system must include notifications.

15.1 Notification types

At minimum: - listing approved - listing rejected - new message - report status changed - payment successful - promotion activated - promotion expired

15.2 Notification behavior

Users must be able to: - view notifications list - mark notification as read - see unread badge/count

15.3 Notification storage

Database-backed notifications are required.

Bonus: - push notifications - email notifications

---

16. Reporting and Moderation Requirements

16.1 Reporting

Users must be able to report: - listings - optionally users - optionally messages for abuse

16.2 Report reasons

Examples: - spam - fake listing - scam risk - duplicate - offensive content - prohibited item - harassment

16.3 Report fields

Suggested: - id - reporter_user_id - target_type - target_id - reason_code - reason_text - status - resolution_note - reviewed_by_admin_id - created_at - reviewed_at

16.4 Admin moderation actions

Admins must be able to: - view reports queue - inspect target object - resolve/dismiss report - reject listing - archive listing - suspend user if needed - write moderation note

16.5 Auditability

Moderation actions should be recorded in audit logs.

---

17. Payments Module Requirements

Payments are mandatory in this specification.

A real payment gateway integration may be mocked or sandboxed, but the system architecture must be production-oriented.

17.1 Payment use cases

Users must be able to pay for: - promotions/boosting/targeting - optional featured listing placement - optional premium features

17.2 Payment model

The project must include: - payment initiation - payment record creation - payment status tracking - success/failure handling - transaction history for user - admin visibility into payments

17.3 Payment statuses

At minimum: - pending - successful - failed - cancelled - refunded (optional but recommended in data model)

17.4 Payment fields

Suggested: - id - user_id - listing_id (nullable if general purchase) - promotion_id/package_id - amount - currency - status - payment_provider - provider_reference - created_at - updated_at - paid_at

17.5 Wallet or balance option

Optional but strong bonus: - internal user balance/wallet - top-up flow - deduct funds for promotions - transaction ledger

17.6 Payment architecture expectations

Even if mocked, system should distinguish: - payment intent/request - provider callback or success confirmation simulation - final business activation after successful payment

Students must not simply toggle a boolean without transaction records.

---

18. Promotions, Boosting, and Targeting Requirements

Promotions are mandatory.

18.1 Promotion goals

Allow users to increase visibility of their listing.

18.2 Required promotion types

At minimum include one or more of: - featured listing - boosted listing - top-of-feed placement - city/category targeting

18.3 Targeting

The platform must support some concept of targeting.

Acceptable forms: - city-based targeting - category-based targeting - time-based boosting duration - audience segment approximation

At minimum, the student must implement: - promotion package selection - target selection (for example city or category) - duration selection - price calculation - payment tie-in - activation after successful payment

18.4 Promotion fields

Suggested: - id - listing_id - user_id - promotion_type - target_city - target_category_id - starts_at - ends_at - status - purchased_price - impressions_limit (optional) - daily_budget (optional)

18.5 Promotion admin controls

Admins must be able to: - view all promotions - create/edit promotion packages - deactivate invalid promotions - inspect active and expired promotions

18.6 Promotion UI expectations

For mobile, user should be able to: - open own listing - choose promote/boost action - choose package - choose target/duration - see price clearly - pay - see active promotion state

---

19. Languages / Localization Requirements

Languages are mandatory.

19.1 Mobile localization

The app must support at least **two languages**.

Recommended examples: - English + Russian - English + Kyrgyz - Russian + Kyrgyz

19.2 What must be localized

At minimum: - navigation labels - buttons - forms - validation messages - main screens - system statuses

19.3 Backend localization considerations

At minimum backend must support: - storing user preferred language - returning machine-readable codes for statuses and enums

Optional but strong: - localized category names - localized system text content in admin-managed tables

19.4 Admin and content language support

Recommended: - categories have localized names - promotion package titles can be localized

Students do not need a perfect enterprise i18n system, but must clearly show real localization support.

---

20. Admin Panel Detailed Requirements

The admin panel must not be superficial.

20.1 Admin authentication

Required: - admin login - protected admin routes/pages - role checks

20.2 Dashboard

Dashboard must include at minimum: - total users - active users - blocked users - total listings - pending listings - approved listings - rejected listings - total conversations - total messages - total reports - total payments - total revenue from promotions - active promotions

Recommended: - daily/weekly charts - new users trend - new listings trend - top categories - top cities

20.3 User management

Admin must be able to: - search users - view user detail - suspend/unsuspend user - inspect user listings - inspect user promotions - inspect user payments - inspect user reports

20.4 Listings moderation

Admin must be able to: - view listings table - filter by status/category/city/owner - open listing detail - approve listing - reject listing with note - archive listing - mark featured manually if policy allows - inspect associated reports

20.5 Messaging oversight

For abuse review purposes, admin should be able to: - inspect conversations tied to reports or abuse claims - not casually intrude into all conversations without reason in UI design

At minimum the data model and admin flow should support moderation investigation.

20.6 Reports management

Admin must be able to: - browse reports - filter by status and reason - open target object - record moderation action - resolve or dismiss report

20.7 Categories management

Admin must be able to: - create category - edit category - enable/disable category - adjust ordering

20.8 Promotions management

Admin must be able to: - view active promotions - view expired promotions - create/edit promotion packages - deactivate promotions if needed

20.9 Payments management

Admin must be able to: - view payment records - filter by status/provider/date - inspect payment details - see linked user/listing/promotion

20.10 Localization/content management

Recommended: - manage category labels in supported languages - manage promotion package labels

---

21. Backend API Requirements

21.1 API quality standards

The backend must: - use REST conventions consistently - validate requests rigorously - return correct status codes - use structured schemas - separate concerns - avoid monolithic spaghetti files

21.2 Suggested route groups

- `/auth`
- `/users`
- `/profile`
- `/public/users`
- `/categories`
- `/listings`
- `/listing-media`
- `/favorites`
- `/conversations`
- `/messages`
- `/attachments`
- `/notifications`
- `/reports`
- `/payments`
- `/promotions`
- `/admin/...`

21.3 API patterns

Required: - pagination support - filtering support - sorting support - request validation - response schemas - clear error messages

21.4 Error handling

Must correctly handle cases such as: - invalid credentials - duplicate account fields - forbidden update of another user's listing - invalid promotion purchase - payment failure - messaging non-existent user/listing - unauthorized attachment access - invalid status transitions - uploading unsupported file types - trying to promote unapproved listing

21.5 API documentation

Required: - OpenAPI / Swagger available and usable

Strongly recommended: - README endpoint summary - Postman collection

---

22. Database Design Requirements

22.1 Required tables/entities

At minimum the project should include logical entities equivalent to: - users - roles or user_role - categories - listings - listing_images/media - favorites - conversations - messages - message_attachments - notifications - reports - payments - promotions - promotion_packages - admin_audit_logs

Optional but strong: - wallets - wallet_transactions - localized_category_translations - user_blocks

22.2 Constraints and indexes

Required examples: - unique email and/or phone - unique favorite(user_id, listing_id) - indexed listing status - indexed category and city - indexed conversation participants - indexed payment status - indexed promotion status

22.3 Timestamps

Most important tables should include: - created_at - updated_at

22.4 Soft delete

Recommended for: - listings - users - messages if needed

22.5 Migrations

Strongly recommended and effectively expected.

---

23. Flutter Mobile App Detailed Requirements

23.1 Required screens

The mobile app should include at minimum: - splash / app initialization - language selection or first-run language support - login - registration - forgot password - home feed - search results - filter modal/screen - listing details - owner public profile - owner listings page - create listing - edit listing - my listings - favorites - inbox/conversations list - conversation detail with attachments - notifications list - profile/settings - payments/promotions history - promote listing flow

23.2 Mobile UX expectations

The app must handle: - loading states - empty states - network errors - form validation - submission progress - attachment upload progress where possible - image placeholders/fallbacks

23.3 Mobile architecture expectations

Should demonstrate: - reusable widgets - separation of API/data/domain/UI layers where reasonable - configuration management - environment-based base URL - maintainable navigation structure

23.4 Quality expectations

The UI should feel intentional and not chaotic.

Must avoid: - broken spacing everywhere - unreadable screens - unclear button meaning - no error feedback - inconsistent naming and navigation

---

24. Security and Production Readiness Requirements

24.1 Authentication security

- hash passwords properly
- never store plain passwords
- protect privileged routes

24.2 Authorization security

- server-side ownership checks
- admin checks
- attachment access control
- message access control

24.3 Config security

- use environment variables
- no secrets committed
- sample env file only

24.4 File security

- validate uploads
- sanitize filenames or generate safe names
- enforce size and type rules

24.5 Common forbidden mistakes

- plain text passwords
- no ownership checks
- open admin routes
- direct file path exposure without control
- trusting frontend only for permissions

---

25. Non-Functional Requirements

25.1 Code quality

Code must be: - readable - modular - named clearly - not full of duplication - maintainable by another engineer

25.2 Performance expectations

Not internet-scale, but should avoid obvious problems such as: - no pagination - N+1 style careless querying everywhere - giant unbounded lists - duplicate API calls on every rebuild

25.3 Reliability

Normal user flows should not crash.

25.4 Observability

Recommended: - structured logs - basic error logging - admin audit logging

---

26. Testing Expectations

Testing is strongly recommended.

26.1 Backend

Recommended tests: - auth flow - listing CRUD - messaging permissions - payment activation logic - promotion creation logic

26.2 Flutter

Recommended tests: - state logic - validation - key widget flows

Even limited tests improve evaluation.

---

27. README and Submission Requirements

Students must submit: - source code - README - migrations/schema - sample env file - demo credentials - screenshots or demo video

27.1 README must include

- project overview
- chosen marketplace domain
- architecture explanation
- database explanation
- setup instructions

- backend run steps
- mobile run steps
- admin run steps
- environment variables explanation
- demo admin credentials
- demo user credentials
- payment/promotion assumptions
- localization support explanation
- known limitations
- future work

27.2 Strongly recommended additions

- ER diagram
- sequence diagrams for payment/promotion flow
- Postman collection
- Docker Compose

27.3 Demo Video (Mandatory)

Students must record a demo video and include a link to it in the README.

Requirements:

- Duration: 3–10 minutes
- Must be screen recording with voice explanation
- Must be accessible via link (YouTube, Google Drive, etc.)

The video must demonstrate ALL implemented functionality, including:

Core flows

- User registration and login
- Profile update
- Creating a listing
- Editing a listing
- Viewing own listings
- Viewing another user's public profile
- Viewing listings by a specific owner

Marketplace functionality

- Browsing listings
- Search
- Filters and sorting
- Listing details screen

Messaging

- Starting a conversation
- Sending messages

- Sending attachments (image/document)
- Viewing conversation history

Favorites

- Adding/removing favorites
- Viewing favorites list

Promotions & payments

- Selecting a promotion package
- Choosing targeting (city/category/etc.)
- Payment flow (mock or real)
- Activation of promotion

Admin panel

- Admin login
- Viewing dashboard
- Moderating listings (approve/reject)
- Viewing reports
- Managing users
- Viewing payments/promotions

Localization

- Switching language in the app
- Showing UI changes

Edge cases (at least a few)

- validation errors
- empty states
- error handling

Important rule

If a feature exists in the code but is not shown in the demo video, it may be treated as not implemented.

---

28. Required Deliverables

At minimum, final submission must contain: 1. Flutter mobile app 2. FastAPI backend 3. MySQL database schema/migrations 4. Admin panel 5. Documentation 6. Demo credentials 7. Seed/demo data

Strong bonus: 8. Dockerized setup 9. Deployed demo 10. Tests 11. API collection

---

29. Evaluation Rubric

29.1 Functionality — 25%

- Are the important product flows working end-to-end?
- Are CRUD, messaging, payments, promotions, and admin flows real?

29.2 Architecture and code quality — 20%

- Is the code organized?
- Are responsibilities separated well?
- Is the design maintainable?

29.3 Backend and database design — 15%

- Are APIs clean and validated?
- Is the DB model thoughtful?
- Are business rules enforced?

29.4 Mobile quality — 15%

- Is the app polished and usable?
- Does it support all required product flows?

29.5 Admin panel quality — 10%

- Is admin actually operationally useful?
- Are moderation and control flows complete?

29.6 Production readiness thinking — 10%

- Payments modeled properly
- promotions modeled properly
- messaging attachments handled safely
- localization considered correctly

29.7 Documentation and delivery — 5%

- Can evaluators run and understand it?

---

30. Minimum Pass vs Strong vs Exceptional

Minimum pass

A student should not pass with only basic CRUD.

At a minimum, a passing submission should include: - auth - listing CRUD - owner public page - owner listings page - favorites - messaging - message attachments - admin moderation - payment records - promotion purchase flow - at least two-language support

Strong submission

A strong student will additionally show: - clear architecture - robust validations - good admin UX - thoughtful schema - correct permission checks - real pagination/filtering - coherent promotion/payment logic

Exceptional submission

An exceptional student will also show: - deployment - Docker - tests - richer analytics - wallet support - translation tables - strong audit logging - polished mobile UX - careful trade-off explanations

---

31. Rules and Integrity

Students may use: - open source libraries - starter setup code - AI tools for assistance

Students may not: - submit copied marketplace templates as-is - submit code they cannot explain - fake working features with static screens only - omit the backend logic for critical flows such as payments/promotions while pretending it exists

If AI tools are used, students must be able to explain: - architecture decisions - schema decisions - permission model - payment flow model - messaging design

---

32. Suggested Follow-Up Interview Questions

Reviewers may ask: - Why did you structure the backend this way? - Why did you choose this Flutter state management? - How do you enforce message attachment security? - How would you support real payment callbacks? - How would you redesign promotions at scale? - How would you localize dynamic category attributes? - Why did you choose this schema for conversations/messages? - What would you improve in one more week? - What are the biggest production risks in your current version?

---

33. Final Instruction to Students

This project is intentionally ambitious.

Do not optimize only for the number of features. Optimize for: - correctness - completeness of major flows - sound engineering decisions - clean architecture - secure handling of data and permissions - and a submission that reviewers can trust

A serious, coherent, well-executed product is the goal.

Your submission should make the reviewer think: **“This student can already build real systems with guidance and can grow into production work fast.”**